Imperial College
London

Lecture 8

Chip Testing

Prof Peter YK Cheung

Department of Electrical & Electronic Engineering

URL: https://elec70142-digital-vlsi-design.github.io/course-website/
E-mail: p.cheung@imperial.ac.uk

Important of Testing

- Testing is one of the most expensive parts of chips
  - Logic verification accounts for > 50% of design effort for many chips
  - Debug time after fabrication has enormous opportunity cost
  - Shipping defective parts can sink a company
- Example: Intel FDIV bug
  - Logic error not caught until > 1M units shipped
  - Recall cost $450M (!!!)

VLSI Tests

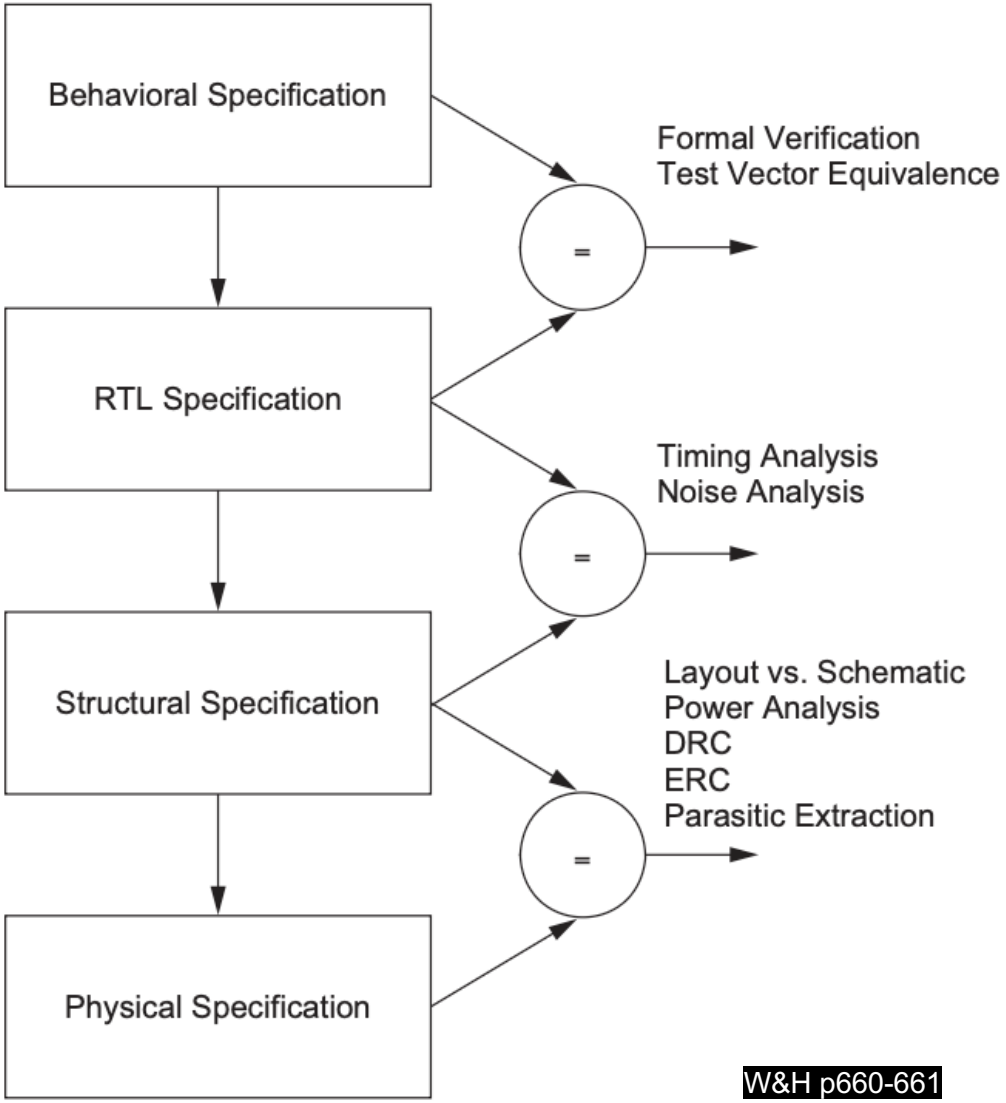
1. Functional test
2. Silicon debug
3. Manufacturing tests

Stage when defect found	Cost per device
Wafer	$0.01 - $0.1
Packaged chip	$0.1 - $1
Board	$1 - $10
System	$10 - $100
Field	$100 - $1000

W&H p659-660

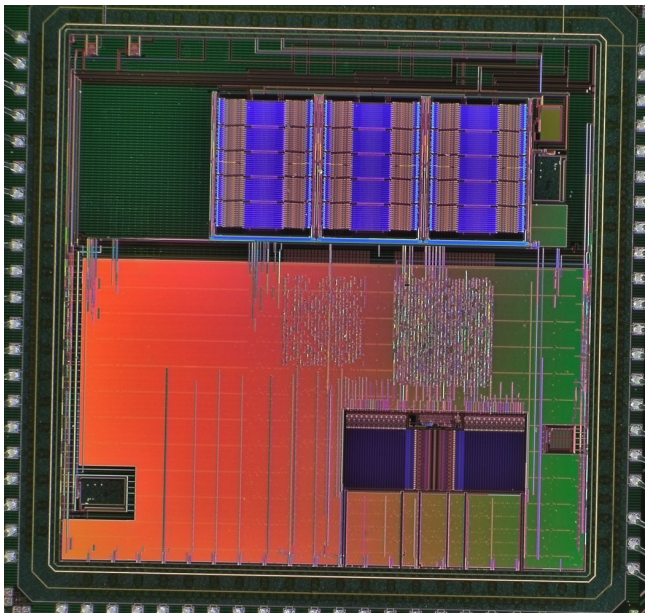
Functional/Logic Verification

- Does the chip simulate correctly?
  - Usually done at HDL level
  - Verification engineers write test bench for HDL
    - ➢ Can't test all cases
    - ➢ Look for corner cases
    - ➢ Try to break logic design
- Ex: 32-bit adder
  - Test all combinations of corner cases as inputs:
    - ➢ 0, 1, 2, 2^{31}-1, -1, -2^{31}, a few random numbers
- Good tests require ingenuity and deep understanding of the circuit



Silicon Debug

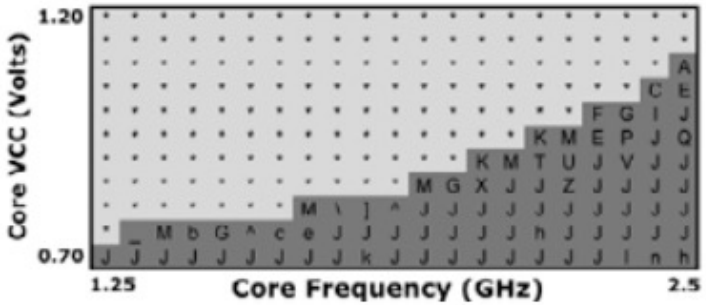
- Test the first chips back from fabrication
  - If you are lucky, they work the first time
  - If not…
- Logic bugs vs. electrical failures
  - Most chip failures are logic bugs from inadequate simulation
  - Some are electrical failures
    - Crosstalk
    - Dynamic nodes: leakage, charge sharing
    - Ratio failures
  - A few are tool or methodology failures (e.g. DRC)
- Fix the bugs and fabricate a corrected chip



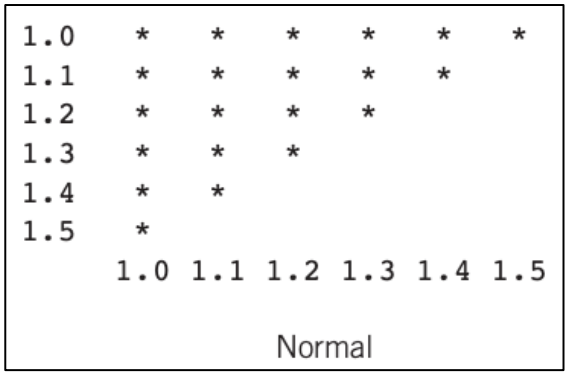
W&H p662-664

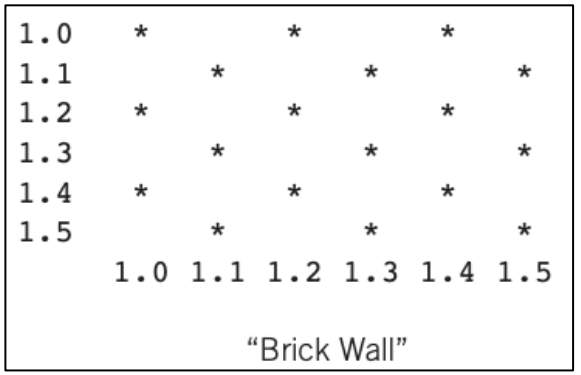
Shmoo Plots

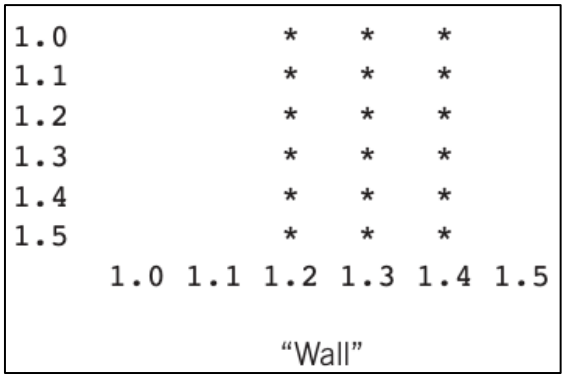
- How to diagnose failures?
  - Hard to access chips
    - Picoprobes
    - Electron beam
    - Laser voltage probing
    - Built-in self-test
- Shmoo plots
  - Vary voltage, frequency
  - Look for cause of electrical failures

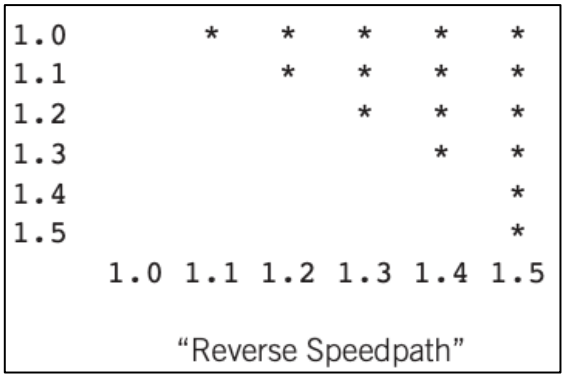


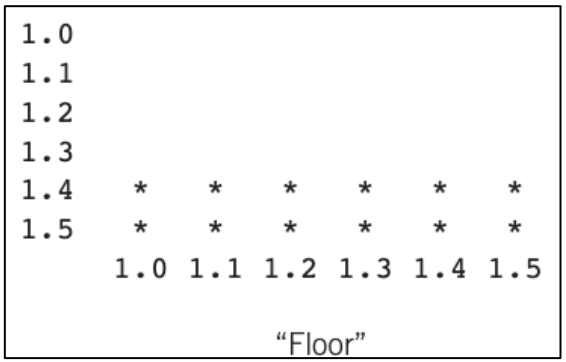
W&H p675-676

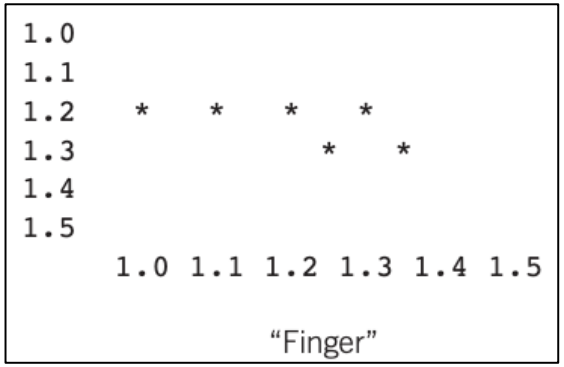






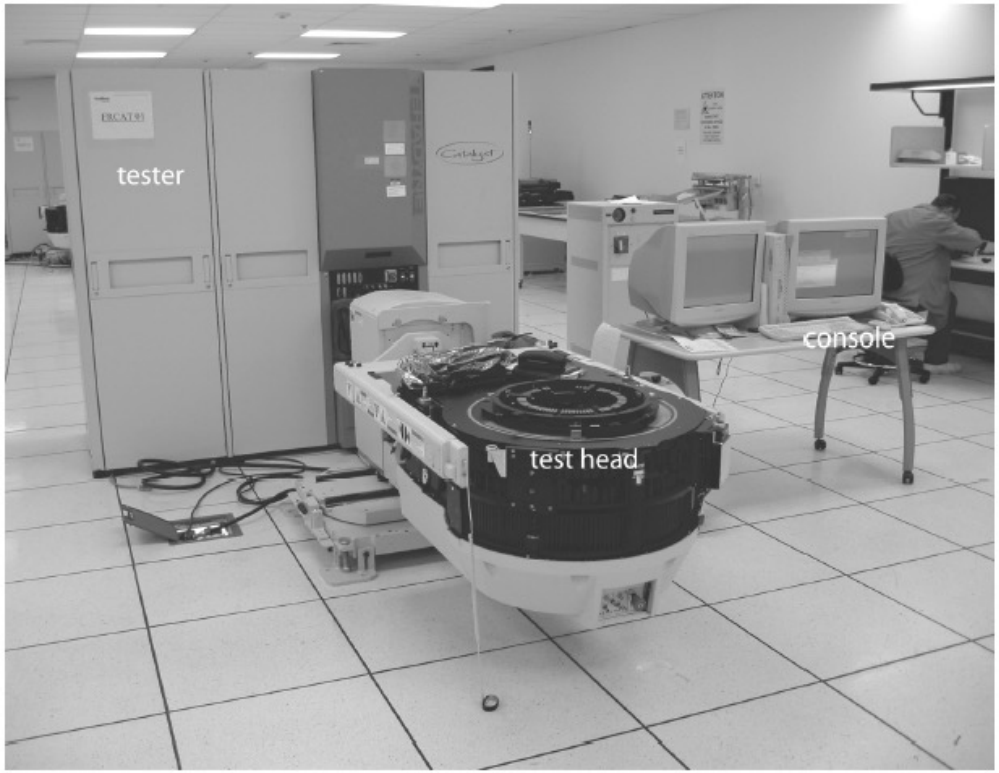






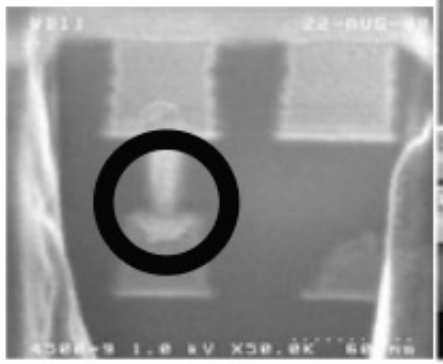
Manufacturing Test

- A speck of dust on a wafer is sufficient to kill chip
- *Yield* of any chip is < 100%
  - Must test chips after manufacturing before delivery to customers and only ship good parts
- Manufacturing testers are very expensive
  - Minimize time on tester
  - Careful selection of *test vectors*

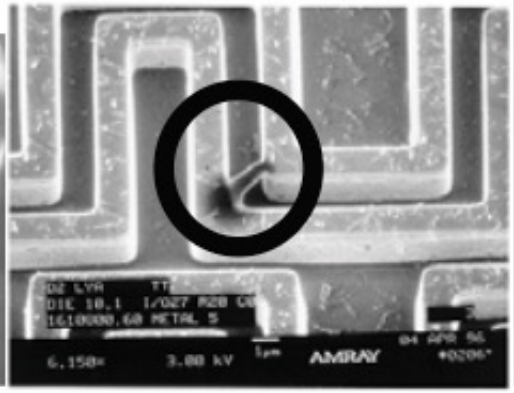


W&H p666

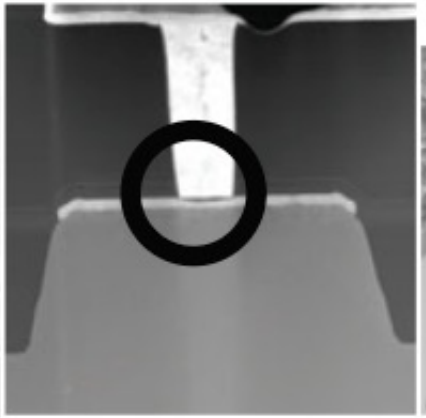
SEM images of manufacturing defects



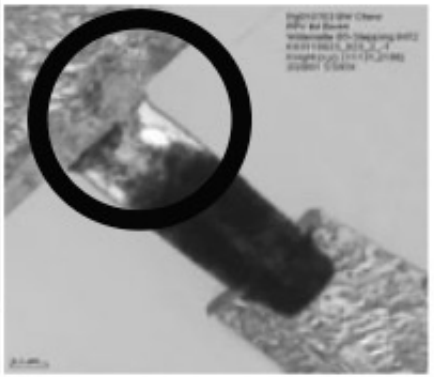
Metal1 Shelving



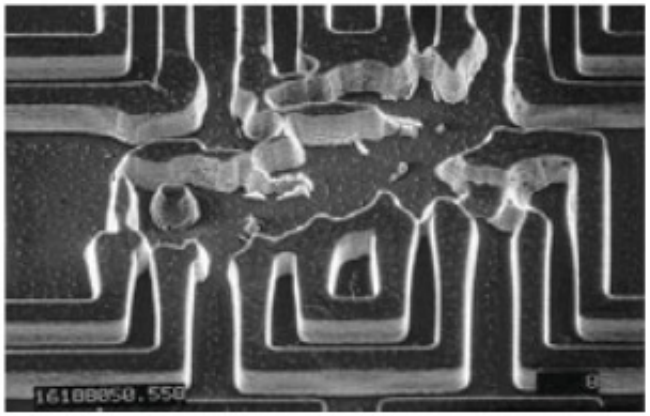
Metal5 Film Particle
(bridging defect)



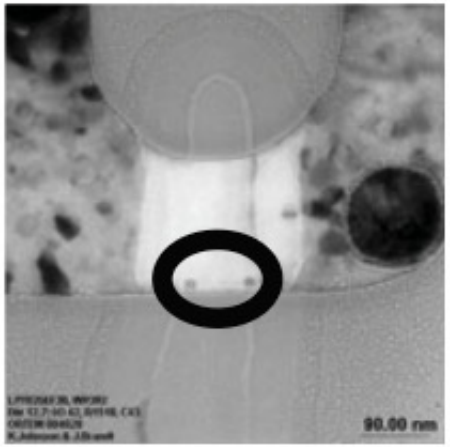
Open Defects



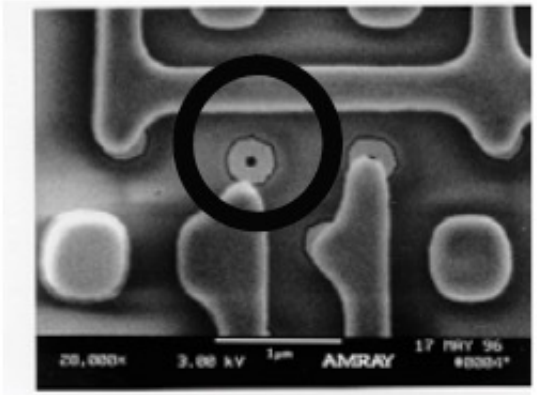
Spongy via2
(infant mortality)



Metal5 Blocked Etch
(patterning defect)



Spot Defects

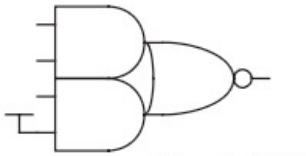


Metal1 Missing Pattern
(open at contact)

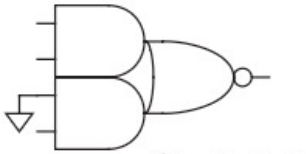
W&H p676-677

Fault Modeling

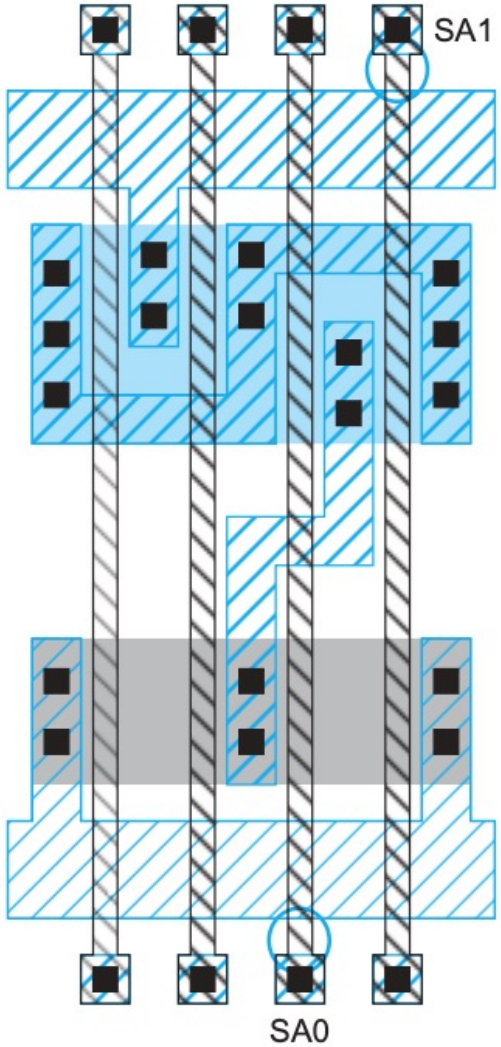
- Manufacturing faults can be due to shorts between two nodes or opens in a circuit in a wire
- This can cause very complicated behaviour
- Stuck-At model:
  - Assume all failures cause nodes to be “stuck-at” 0 or 1, i.e. shorted to GND or VDD
  - Not quite true, but works well in practice



Stuck-At-1
SA1 Fault



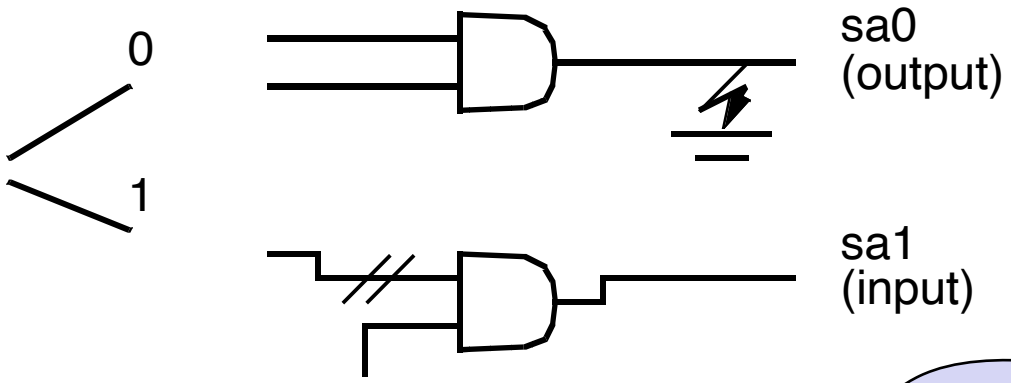
Stuck-At-0
SA0 Fault



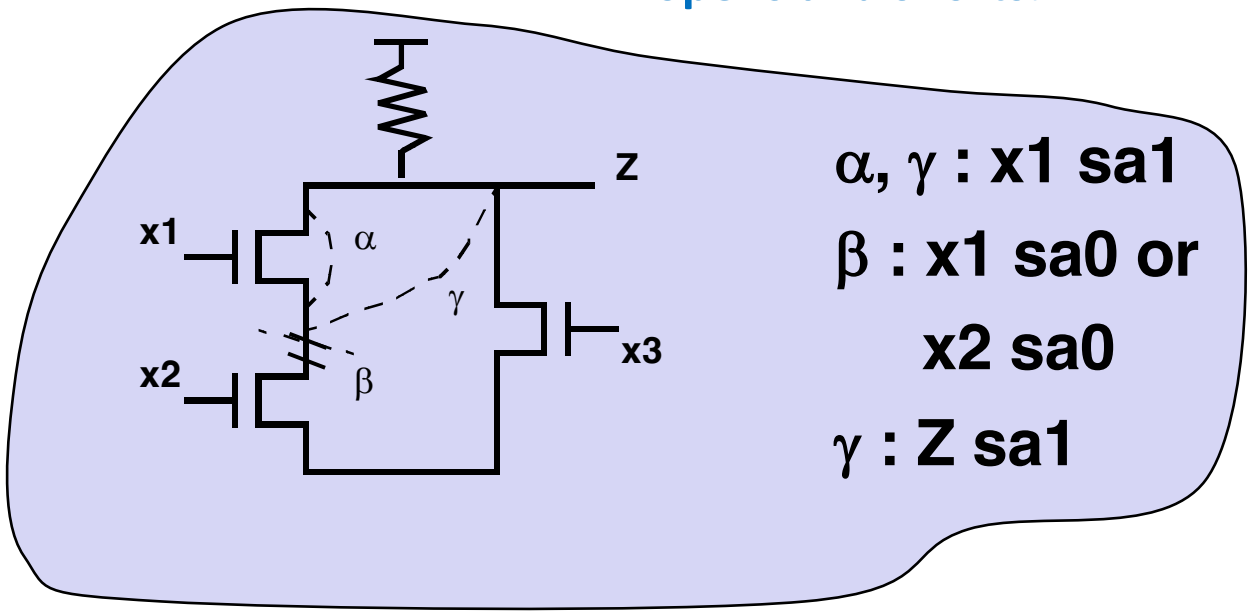
W&H p677

Stuck-at model covering other faults

Most Popular - "Stuck - at" model



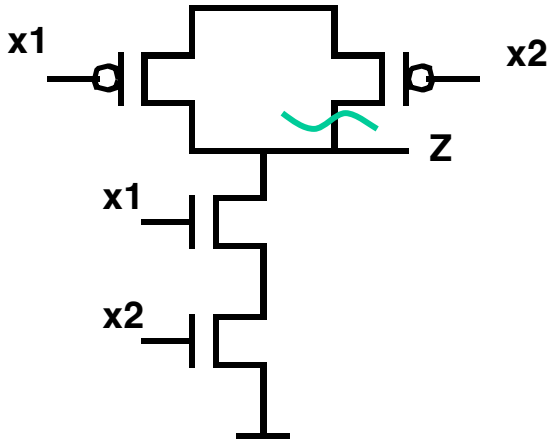
Covers almost all (other) occurring faults, such as opens and shorts.



Limitation of stuck-at model: CMOS open fault



x1	x2	Z
o	x	1
1	1	0
1	0	Z_{n-1}



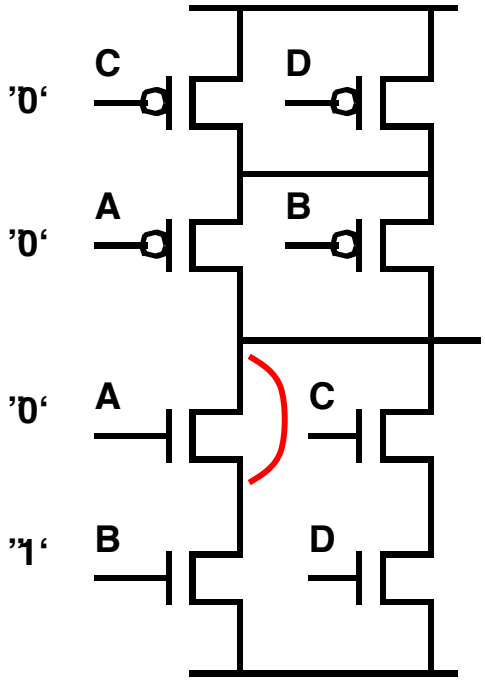
Sequential effect

Needs two vectors to ensure detection!

Other options: use stuck-open or stuck-short models

This requires fault-simulation and analysis at the switch or transistor level - Very expensive!

Limitation of stuck-at model: CMOS short fault



Causes short circuit between
Vdd and GND for A=C=0, B=1

Possible approach:
Supply Current Measurement (IDDQ)
but: not applicable for gigascale integration

Four concepts in testing

In VLSI tests, four concepts are important:

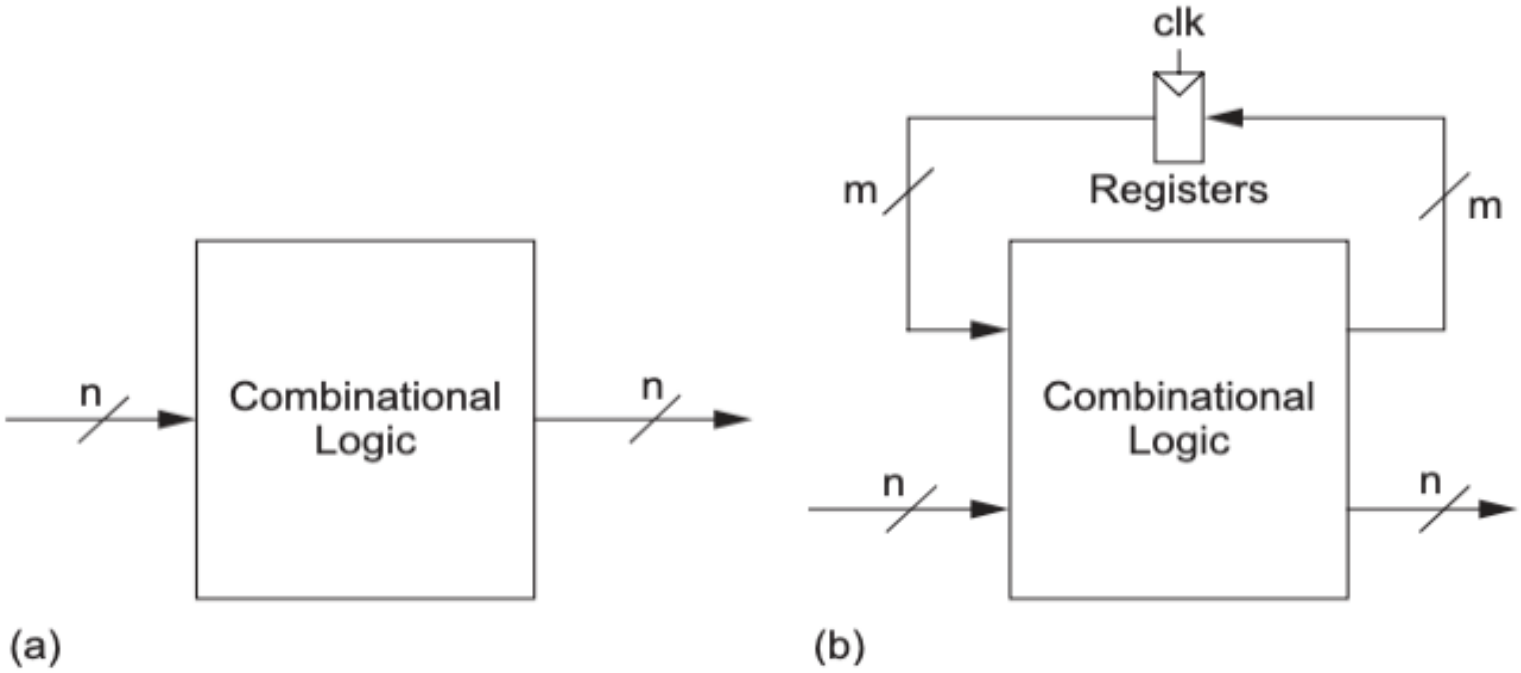
- **Controllability**: measures the ease of setting a node to 0 or 1 state
- **Observability**: measures the degree to which a node's state can be observed
- **Repeatability**: the ability to produce the same outut given the same input
- **Survivability**: the ability to continue functioning after a fault has occurred.

How to test circuits?

- **Combinational Circuits**:
  controllable and observable - relatively easy to determine test patterns
- **Sequential Circuits**: State!
  Turn into combinational circuits or use self-test
- **Memory**: requires complex patterns
  Use self-test

Logic Verification Principles

- Scalability challenge in sequential circuit test.
- N = 25, M = 50.
- Exhaustive test needs around 3.8 x 10^{22} test vectors.
- Assume perform 1 test every microsecond, take 10^{9} years!



W&H p670

Generating and Validating Test-Vectors

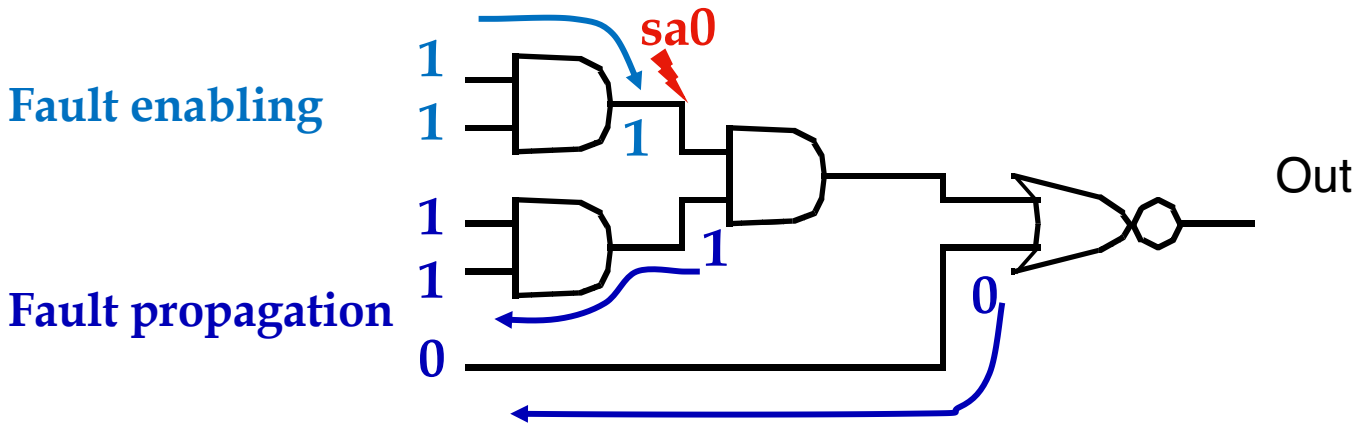
- **Automatic test-pattern generation (ATPG)**
  - for given fault, determine excitation vector (called **test vector**) that will propagate error to primary (observable) output
  - majority of available tools: combinational networks only
  - sequential ATPG is far more complex – normally require extra built-in test circuits
  - CAD tools (e.g. Cadence's Modus tools) are used to automatically produce test vectors to detect stuck-at faults in circuit (see later).
- **Fault simulation**
  - determines **fault coverage** of proposed test-vector set
    - ➢ Fault coverage = no. of faults detectable by test vector set/ total no. of faults
  - world-class quality test coverage >98.5% fault coverage
  - simulates correct network in parallel with faulty networks
- Both require adequate models of faults in CMOS integrated circuits

Automatic Test Pattern Generation (ATPG)

- Test pattern generation is tedious
- Automatic Test Pattern Generation (ATPG) tools produce a good set of vectors for each block of combinational logic
- Scan chains are used to control and observe the blocks
- Complete coverage requires a large number of vectors, raising the cost of test
- Most products settle for covering 90+% of potential stuck-at faults

Path Sensitization

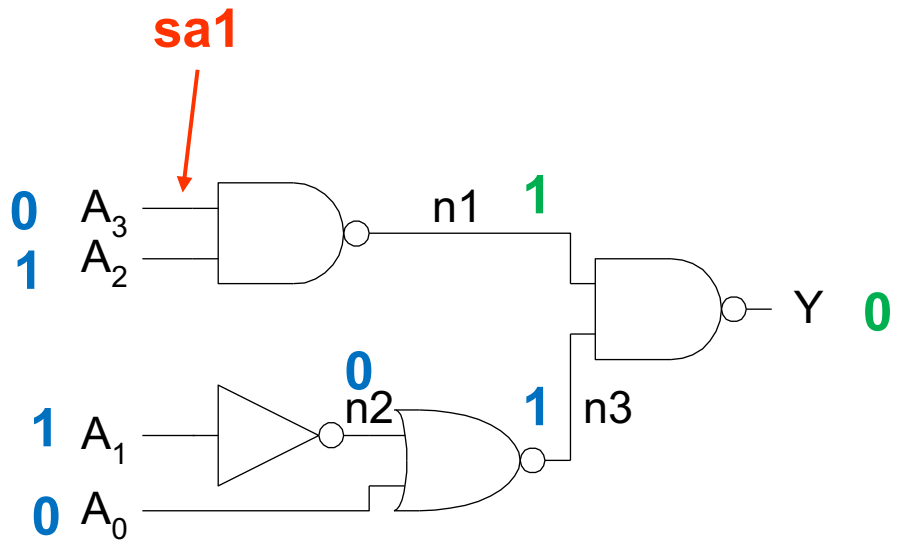
Goals: Determine input pattern that makes a fault controllable (triggers the fault, and makes its impact visible at the output nodes)



Techniques Used: D-algorithm, Podem

Example to generate minimum test vector set

	SA1	SA0
A_3	{0110}	{1110}
A_2	{1010}	{1110}
A_1	{0100}	{0110}
A_0	{0110}	{0111}
N1	{1110}	{0110}
N2	{0110}	{0100}
N3	{0101}	{0110}
Y	{0110}	{1110}



- Minimum set: {0100, 0101, 0110, 0111, 1010, 1110}